

```
returnLatestBlock().currentTransactionHash;
    currentBlock.currentTransactionHash = currentBlock.
calculateBlockDigest();
    this.transactionChain.push(currentBlock);
}
```

In the above code example, we calculate the hash of the previous transaction and the hash of the current transaction before pushing the new block to the blockchain. We also validate the new block before adding it to the blockchain using the method described below.

Validating the blockchain

Each block needs to be validated before it gets added to the blockchain. The validation we used in our implementation is described below:

```
isBlockchainValid() {
    for (let blockCount = 1; blockCount < this.
transactionChain.length; blockCount++) {
        const currentBlockInBlockchain = this.
transactionChain[blockCount];
        const previousBlockInBlockchain = this.
transactionChain[blockCount - 1];
        if (currentBlockInBlockchain.
currentTransactionHash !== currentBlockInBlockchain.
calculateBlockDigest()) {
            return false;
        }

        if (currentBlockInBlockchain.
previousTransactionHash !== previousBlockInBlockchain.
currentTransactionHash) {
            return false;
        }
    }

    return true;
}
```

In this implementation, there are a lot of features missing as of now, like validation of the funds, the rollback feature in case the newly added block corrupts the blockchain, etc. If anyone is interested in tackling fund validation, the rollback or any other issue they find, please go to my GitHub repository, create an issue and the fix for it, and send me a pull request or just fork the repository and use it whichever way this code suits your requirements.

A point to be noted here is that in this implementation, there are numerous ways to tamper with the blockchain. One way is to tamper with the data alone. The implementation for that is done in the branch https://github.com/abhiit89/AngCoins/tree/tampering_data.

Another way is to not only change the data but also update the hash. Even then the current implementation can invalidate it. The code for it is available in the branch https://github.com/abhiit89/AngCoins/tree/tampering_data_with_updated_hash.

Proof of work

With the current implementation, it is still possible that someone can spam the blockchain by changing the data in one block and updating the hash in all the following blocks in the blockchain. In order to prevent that, the concept of the 'proof of work' suggests a difficulty or condition that each block that is generated has to meet before getting added to the blockchain. This difficulty prevents very frequent generation of the block, as the hashing algorithm used to generate the block is not under the control of the person creating the block. In this way, it becomes a game of hit and miss to try to generate the block that meets the required conditions.

For our implementation, we have set the difficult task that each block generated must have two '00' in the beginning of the hash, in order to be added to the blockchain. For example, we can modify the function to add a new block to include the difficult task, given as below:

```
addNewTransactionBlockToTransactionChain(currentBlock) {
    currentBlock.previousTransactionHash = this.
returnLatestBlock().currentTransactionHash;
    currentBlock.mineNewBlock(this.difficulty);
    this.transactionChain.push(currentBlock);
}
```

This calls the mining function (which validates the difficult conditions):

```
mineNewBlock(difficulty) {
    while(this.currentTransactionHash.substring(0,
difficulty) !== Array(difficulty + 1).join('0')) {
        this.nonce++;
        this.currentTransactionHash = this.
calculateBlockDigest();
    }
    console.log('New Block Mined --> ' + this.
currentTransactionHash);
}
```

The complete code for this implementation can be seen in the branch https://github.com/abhiit89/AngCoins/tree/block_chain_mining.

Blockchain providers

Blockchain technology, with its unprecedented way of managing trust and data and of executing procedures, can transform businesses. Here are some open source blockchain platforms.

Continued on page 103...